Limitations of *k*-means Clustering

After seeing the Lloyd algorithm in action, it may seem that clustering is easy. If you think so, consider the following question.

STOP and Think: How would you cluster the points in the figure below?

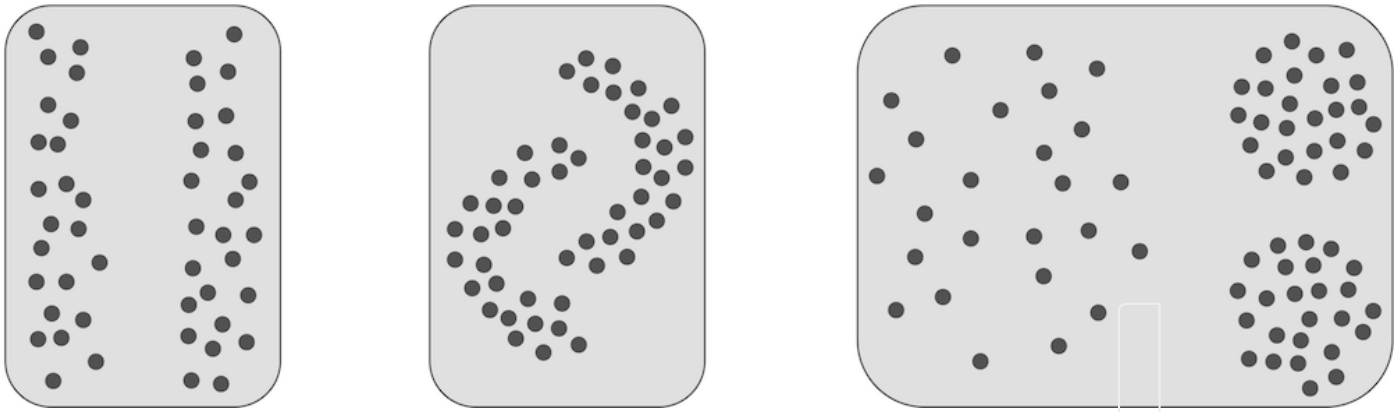


Figure: Difficult clustering problems for $k = 2$ (left and middle) and $k = 3$ (right).

Limitations of k-means Clustering | Step 2

In the case of challenging clustering problems, the Lloyd algorithm sometimes fails to identify what may seem like obvious clusters (see figure below).

STOP and Think: The Lloyd algorithm assigns each data point to its closest center, with ties broken arbitrarily. What are the negative effects of this assignment?

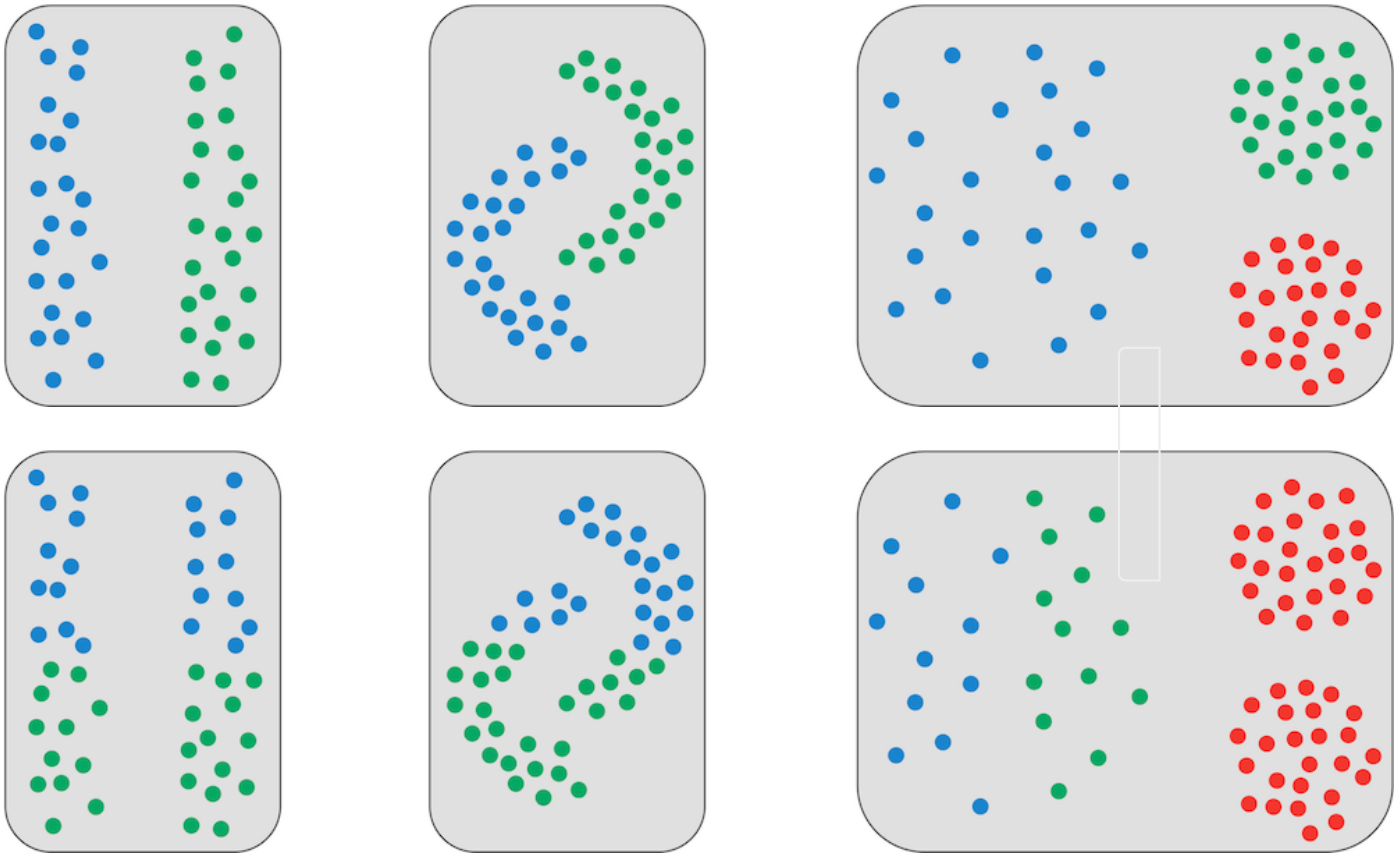


Figure: The human eye (top) and the Lloyd algorithm (bottom) often disagree in the case of elongated clusters (left), clusters with non-globular shapes (middle), and clusters with widely different data point densities (right).

Limitations of k-means Clustering | Step 3

One weakness with our formulation of the *k*-means Clustering Problem is that it forces us to make a “hard” assignment of each point to only one cluster. This strategy makes little sense for **midpoints**, or points that are approximately equidistant from two centers (see figure below).

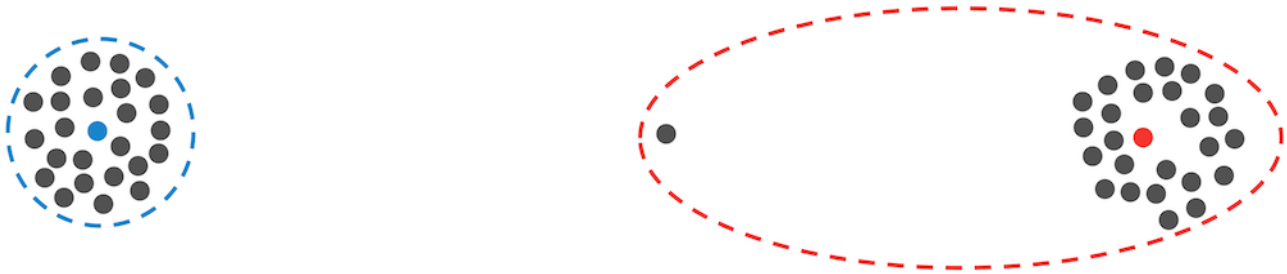


Figure: The Lloyd algorithm rigidly assigns the midpoint between the two clusters to a single cluster, rather than giving it approximately equal membership in each cluster.

Limitations of k-means Clustering | Step 4

Our goal is to transition away from a rigid assignment of a data point to a single cluster (figure below (left)) and toward a “soft” assignment (figure below (right)).

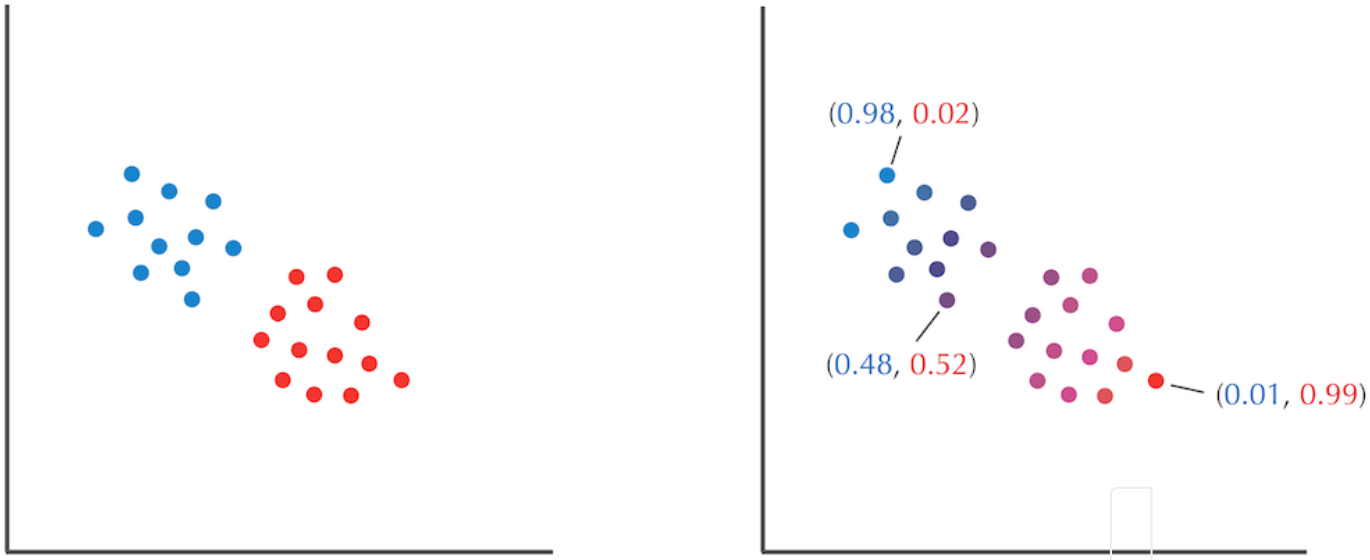


Figure: (Left) Points partitioned into two clusters by the Lloyd algorithm; points are colored red or blue depending on their cluster membership. (Right) We can visualize a soft clustering of the same data into two clusters as assigning each point a pair of numbers representing the point’s percentage of “blue” and “red” based on each cluster’s “responsibility” for this point. The colors mix to form a color from the red-blue spectrum.

Flipping coins with unknown biases

To develop an algorithm for soft clustering, we will introduce a seemingly unrelated analogy. Tom Stoppard's play *Rosencrantz and Guildenstern Are Dead* opens with the two title characters flipping a coin over and over, finding that it results in "heads" 157 consecutive times. Rosencrantz and Guildenstern question whether they have become detached from the laws of probability, but if you witnessed such an event, you would probably guess that the coin was biased. Say that your friend flips a biased coin n times, and you would like to estimate the probability θ that a single flip of this coin results in heads.

STOP and Think: For each value of θ between 0 and 1, you can compute the probability of a given sequence of flips. How would you estimate the value of θ maximizing this probability after watching a series of flips where the coin lands on heads i out of n times?

From Coin Flipping to k-Means Clustering | Step 2

It seems like the best estimate of θ should be the number of occurrences of heads divided by the total number of coin flips. But how can we prove that this is the case? Given a sequence of n coin flips containing i heads, the probability that a coin with bias θ generated this sequence is $f(\theta) = \theta^{i} \cdot (1 - \theta)^{n - i}$. Since the most likely coin bias is the value of θ that maximizes this probability, we will set the derivative of $f(\theta)$ equal to zero:

$$\begin{aligned} f'(\theta) &= i \cdot \theta^{i-1} \cdot (1 - \theta)^{n-i} - \theta^{i} \cdot (n - i) \cdot (1 - \theta)^{n - i - 1} \\ &= (i \cdot (1 - \theta) - \theta \cdot (n - i)) \cdot \theta^{i-1} \cdot (1 - \theta)^{n - i - 1} = 0. \end{aligned}$$

As expected, the solution of this equation is $\theta = i/n$, implying that the observed proportion of heads provides the best estimate for θ.

From Coin Flipping to k-Means Clustering | Step 3

To make the coin flipping problem a bit more interesting, suppose that your friend secretly switches between two coins *A* and *B* that look identical but have unknown biases θ_A and θ_B. After observing a sequence of coin flips, your goal is to estimate θ_A and θ_B, which we collectively denote by *Parameters*.

We will simplify the problem by assuming that every *n* flips, your friend secretly decides to either keep the same coin or switch coins. Five sequences of *n* = 10 flips are shown in the figure below. We will represent the proportion of heads in each of these sequences as a vector,

$$Data = (Data_1, Data_2, Data_3, Data_4, Data_5) = (0.4, 0.9, 0.8, 0.3, 0.7)$$

If you knew that your friend used coin *A* in the first and fourth sequences of flips, then you would estimate θ_A by computing the proportion of heads in these sequences,

$$\theta_A = \frac{Data_1 + Data_4}{2} = \frac{0.4 + 0.3}{2} = 0.35$$

You would then estimate θ_B as the proportion of heads in the remaining three sequences,

$$\theta_B = \frac{Data_2 + Data_3 + Data_5}{3} = \frac{0.9 + 0.8 + 0.7}{3} = 0.8$$

										Data
H	T	T	T	H	T	T	H	T	H	0.4
H	H	H	H	T	H	H	H	H	H	0.9
H	T	H	H	H	H	H	T	H	H	0.8
H	T	T	T	T	T	H	H	T	T	0.3
T	H	H	H	T	H	H	H	T	H	0.7

Figure: Five sequences of ten coin flips result in *Data* = (0.4, 0.9, 0.8, 0.3, 0.7). “H” denotes heads and “T” denotes tails.

We will represent this choice of coins as a binary vector *HiddenVector* = (1, 0, 0, 1, 0), where a 1 in the *k*-th position denotes that coin *A* was used to generate the *k*-th sequence of flips, and a 0 denotes that coin *B* was used. This notation allows us to rewrite the equations for *Parameters* in terms of *Data* and *HiddenVector*:

$$\theta_A = \frac{\sum_i HiddenVector_i \cdot Data_i}{\sum_i HiddenVector_i} = \frac{1 \cdot 0.4 + 0 \cdot 0.9 + 0 \cdot 0.8 + 1 \cdot 0.3 + 0 \cdot 0.7}{1 + 0 + 0 + 1 + 0}$$
$$\theta_B = \frac{\sum_i (1 - HiddenVector_i) \cdot Data_i}{\sum_i (1 - HiddenVector_i)} = \frac{0 \cdot 0.4 + 1 \cdot 0.9 + 1 \cdot 0.8 + 0 \cdot 0.3 + 1 \cdot 0.7}{0 + 1 + 1 + 0 + 1}$$

where *i* runs over all data points.

The expression $\sum_i HiddenVector_i \cdot Data_i$ is the **dot product** of vectors *HiddenVector* and *Data*, written *HiddenVector* · *Data*. Define the **all ones vector**, written $\vec{1}$, as the vector consisting of all ones and with length equal to *HiddenVector*. This allows us to write $\sum_i HiddenVector_i$ as the dot product *HiddenVector* · $\vec{1}$ and $\sum_i (1 - HiddenVector_i)$ as the dot product ($\vec{1}$ - *HiddenVector*) · $\vec{1}$. As a result, the above equations become

$$\theta_A = \frac{HiddenVector \cdot Data}{HiddenVector \cdot \vec{1}}$$
$$\theta_B = \frac{(\vec{1} - HiddenVector) \cdot Data}{(\vec{1} - HiddenVector) \cdot \vec{1}}$$

STOP and Think: We just saw that given *Data* and *HiddenVector*, we can find *Parameters* = (θ_A, θ_B). If you are given *Data* and *Parameters*, can you find the most likely choice of *HiddenVector*?

From Coin Flipping to k-Means Clustering | Step 5

If we know *Parameters*, then deciding on the most likely choice of *HiddenVector* corresponds to determining whether coin *A* or coin *B* was more likely to have generated the *n* observed flips in each of the five coin flipping sequences. For example, suppose we know that *Parameters* = (θ_A , θ_B) = (0.6, 0.82). If coin *A* was used to generate the fifth sequence of flips (with seven heads and three tails), then the probability that coin *A* generated this outcome is

$$\theta_A^7(1 - \theta_A)^3 \approx 0.00179.$$

If coin *B* was used to generate the fifth sequence, then the probability that it generated this outcome is

$$\theta_B^7(1 - \theta_B)^3 \approx 0.00145.$$

Since 0.00179 > 0.00145, we would set $HiddenVector_5$ equal to 1.

Exercise Break: Determine the rest of the entries in *HiddenVector* for *Parameters* = (0.6, 0.82) and the five sequences of coin flips, reproduced below.

										Data
H	T	T	T	H	T	T	H	T	H	0.4
H	H	H	H	T	H	H	H	H	H	0.9
H	T	H	H	H	H	H	T	H	H	0.8
H	T	T	T	T	T	H	H	T	T	0.3
T	H	H	H	T	H	H	H	T	H	0.7

From Coin Flipping to k-Means Clustering | Step 6

More generally, let

$$Pr(Data_i|\theta) = \theta^{n \cdot Data_i}(1 - \theta)^{n \cdot (1 - Data_i)}.$$

If $Pr(Data_i|\theta_A) > Pr(Data_i|\theta_B)$, then we guess that coin *A* was used in the *i*-th sequence of flips and set $HiddenVector_i$ equal to 1. If $Pr(Data_i|\theta_A) < Pr(Data_i|\theta_B)$, then we guess that coin *B* was used and set $HiddenVector_i$ equal to 0. Ties are broken arbitrarily.

In summary, if *HiddenVector* is known and *Parameters* is unknown, then we can reconstruct the most likely *Parameters* = (θ_A,θ_B):

$$(Data, HiddenVector, ?) \rightarrow Parameters$$

Likewise, if *Parameters* is known and *HiddenVector* is unknown, then we can reconstruct the most likely *HiddenVector*:

$$(Data, Parameters, ?) \rightarrow HiddenVector$$

Our original problem, however, was that both *HiddenVector* and *Parameters* are unknown:

$$(Data, ?, ?) \rightarrow ???$$

Where is the computational problem?

You may have noticed that we have not formulated the computational problem that we are trying to solve. So define

$$\Pr(Data_i|HiddenVector, Parameters) = \begin{cases} \Pr(Data_i|\theta_A) & \text{if } HiddenVector_i = 1 \\ \Pr(Data_i|\theta_B) & \text{otherwise} \end{cases}$$

Furthermore, define

$$\Pr(Data|HiddenVector, Parameters) = \prod_{i=1}^{n} \Pr(Data_i|HiddenVector, Parameters).$$

Given *Data*, the computational problem we are trying to solve is to find *HiddenVector* and *Parameters* maximizing $\Pr(Data|HiddenVector, Parameters)$.

From coin flipping to the Lloyd algorithm

Identifying *HiddenVector* and *Parameters* from *Data* may appear hopeless, but we have already learned that starting from a random guess is not necessarily a bad idea. We will therefore start from an arbitrary choice of *Parameters* = (θ_A,θ_B) and immediately reconstruct the most likely *HiddenVector*:

(*Data*, ?, *Parameters*) → *HiddenVector*

As soon as we know *HiddenVector*, we will question the wisdom of our initial choice of *Parameters* and re-estimate *Parameters*':

(*Data*, *HiddenVector*, ?) → *Parameters*

As illustrated in the figure below for the initial choice of *Parameters* = (0.6, 0.82), we repeat these two steps and hope that *Parameters* and *HiddenVector* are moving closer to the correct values:

$$
\begin{aligned}
(\textit{Data}, ?, \textit{Parameters}) &\rightarrow (\textit{Data}, \textit{HiddenVector}, \textit{Parameters}) \\
&\rightarrow (\textit{Data}, \textit{HiddenVector}, ?) \\
&\rightarrow (\textit{Data}, \textit{HiddenVector}, \textit{Parameters}') \\
&\rightarrow (\textit{Data}, ?, \textit{Parameters}') \\
&\rightarrow (\textit{Data}, \textit{HiddenVector}', \textit{Parameters}') \\
&\rightarrow \quad \ldots
\end{aligned}
$$

Exercise Break: Prove that this process terminates, i.e., that *HiddenVector* and *Parameters* eventually stop changing between iterations.

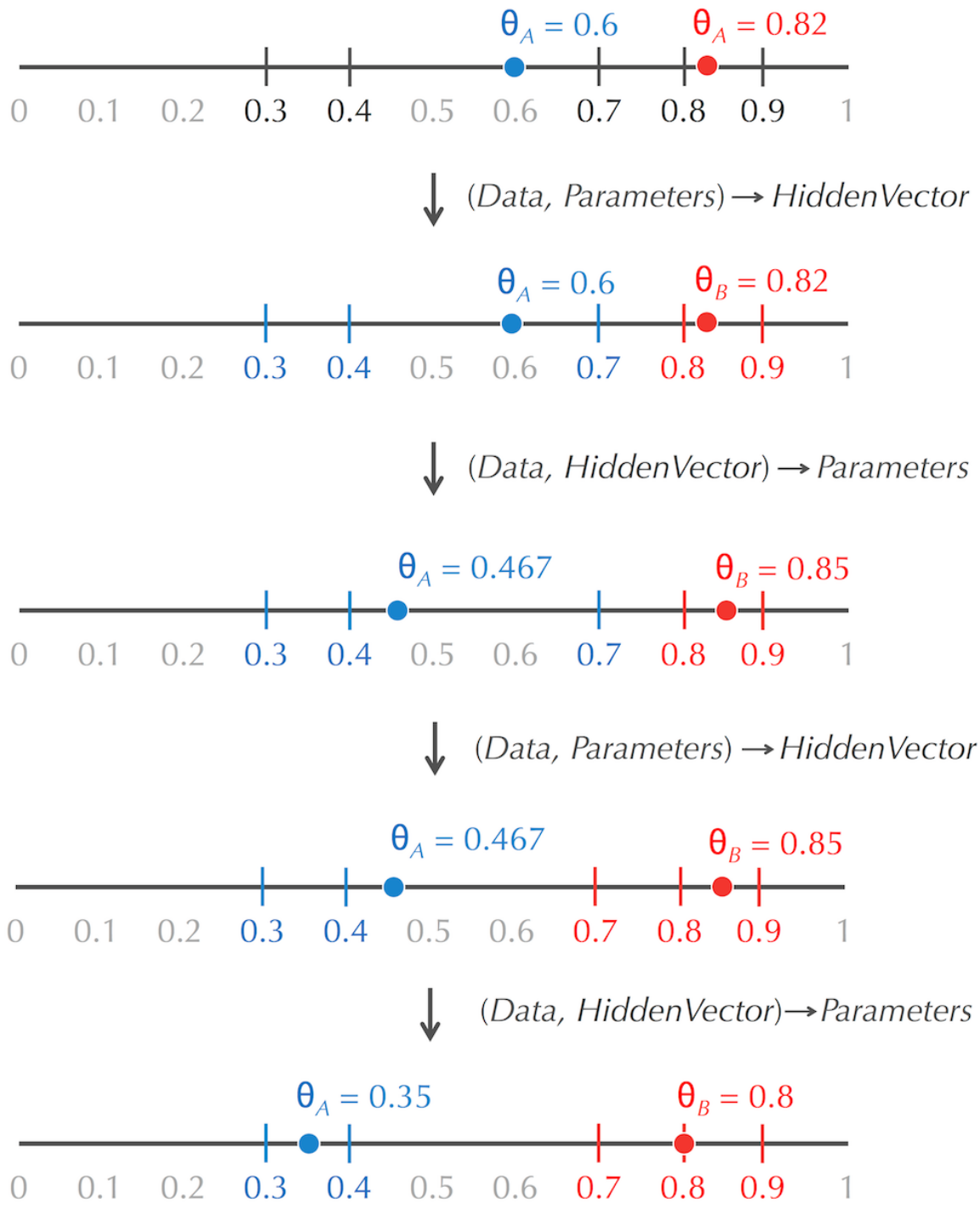


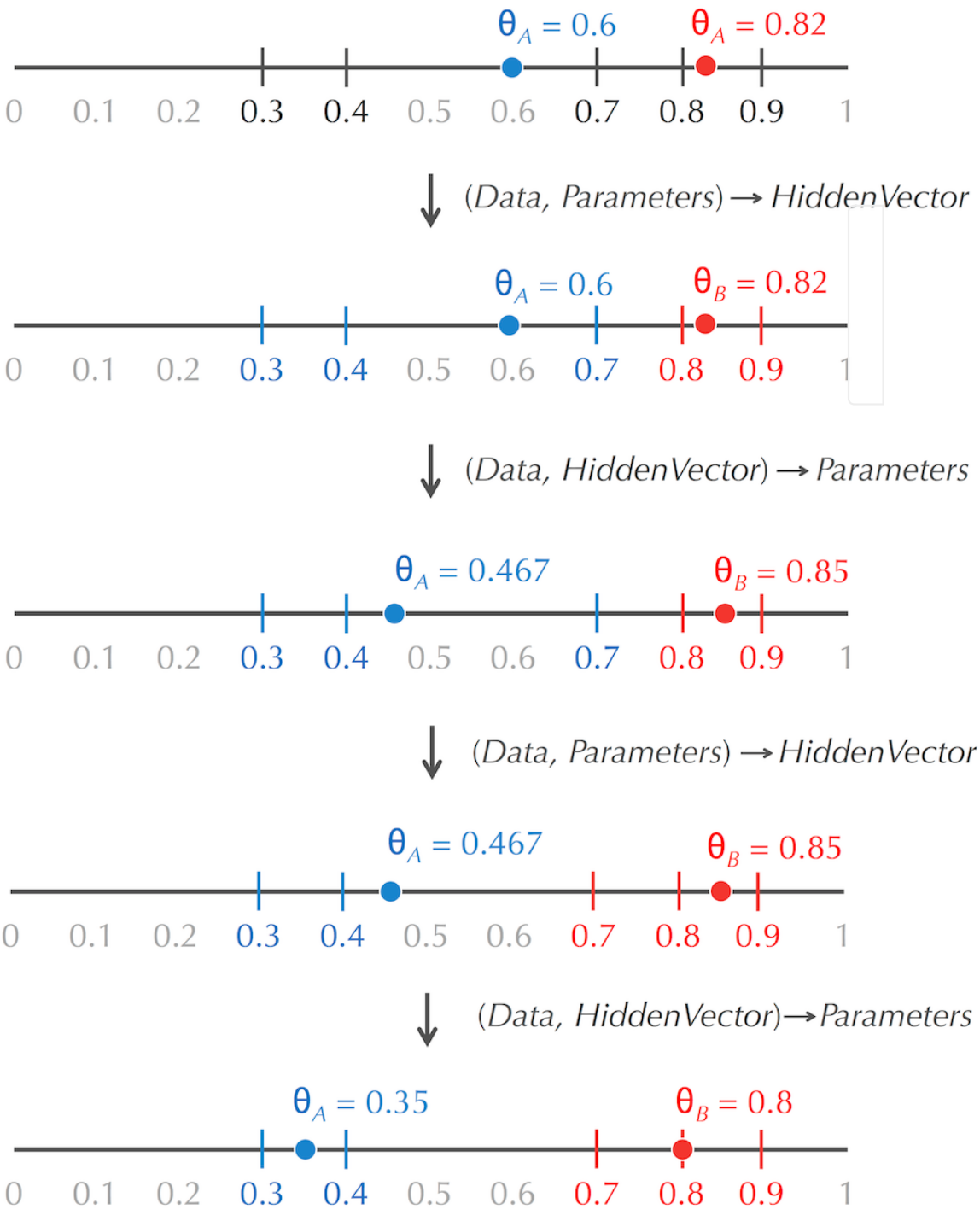
Figure: Starting with *Parameters* = (0.6, 0.82) results in *HiddenVector* = (1, 0, 0, 1, 1) for *Data* = (0.4, 0.9, 0.8, 0.3, 0.7). We then update *Parameters* as (0.467, 0.85), which in turn results in *HiddenVector* = (1, 0, 0, 1, 0). This new vector leads to the assignment of *Parameters* as (0.35, 0.8), at which point the process has terminated because *HiddenVector* will not change in the next step..

STOP and Think: If *HiddenVector* consists of all zeroes, then there are no flips with coin *A*, and the formula for computing θ_A is invalid. What would you do to address this complication?

Return to clustering

The coin flipping figure, reproduced below, has revealed that although our coin flipping analogy seems like a different problem, it is really just a one-dimensional clustering problem in disguise!

STOP and Think: What are *Data*, *HiddenVector*, and *Parameters* in the Lloyd algorithm?



From Coin Flipping to k-Means Clustering | Step 12

Given n data points in m-dimensional space $Data = (Data_1, \ldots, Data_n)$, we represent their assignment to k clusters as an n-dimensional vector

$$HiddenVector = (HiddenVector_1, \ldots, HiddenVector_n),$$

where each $HiddenVector_i$ can take integer values from 1 to k. We will then represent the k centers as k points in m-dimensional space, $Parameters = (\theta_1, \ldots, \theta_k)$. In k-means clustering, similarly to the coin flipping analogy, we are given *Data*, but *HiddenVector* and *Parameters* are unknown. The Lloyd algorithm starts from randomly chosen *Parameters*, and we can now rewrite its two main steps as follows:

- Centers to Clusters: (*Data*, ?, *Parameters*) → *HiddenVector*
- Clusters to Centers: (*Data*, *HiddenVector*, ?) → *Parameters*

From Coin Flipping to k-Means Clustering | Step 13

The only difference between the coin flipping algorithm and the Lloyd algorithm for *k*-means clustering is how they execute the “Centers to Clusters” step. In the former, we compute HiddenVector_i by comparing $Pr(\text{Data}_i|\theta_A)$ with $Pr(\text{Data}_i|\theta_B)$, whereas in the latter, we assign a point to the cluster containing the center nearest to that point.

STOP and Think: Consider the following related questions regarding coin flipping and clustering.

- Is it fair to always select coin *A* if $Pr(Data_i|\theta_A)$ is only slightly larger than $Pr(Data_i|\theta_B)$?
- Is it fair to always assign a point to a center if this center is only slightly closer to the point than another center?

Expectation maximization: the E-step

We will now use our coin flipping analogy to motivate a soft version of *k*-means clustering. Given *Parameters* = (θ_A, θ_B), we can make hard decisions for *HiddenVector* by comparing $Pr(\text{Data}_i|\theta_A)$ with $Pr(\text{Data}_i|\theta_B)$. But this does not mean that we are certain which coin was used. If $Pr(Data_i|\theta_B)$ were approximately equal to $Pr(Data_i|\theta_A)$, then our confidence that coin *B* was used would be approximately 50%. On the other hand, if $Pr(Data_i|\theta_B)$ were much larger than $Pr(Data_i|\theta_A)$, then we would be almost positive that coin *B* was used. More generally, we can speak of our confidence that a coin was used as the "responsibility" of this coin for a given sequence of flips. (The responsibilities should sum to 1.)

In terms of *k*-means clustering, if a data point is a midpoint between two centers, then each of these centers should have about the same responsibility for attracting it to their clusters. As with coin flipping, we demand that the responsibilities of all centers for a given data point sum to 1.

STOP and Think: Given *Parameters* = (θ_A, θ_B) = (0.6, 0.82) and the sequence of coin flips "THHHTHHHTH", how would you compute responsibilities for coins *A* and *B*?

Making Soft Decisions in Coin Flipping | Step 2

To answer the preceding question, we have seen that $\Pr(0.7|\theta_A) = 0.6^7 \cdot 0.4^3 \approx 0.00179$ and that $\Pr(0.7|\theta_B) = 0.82^7 \cdot 0.18^3 \approx 0.00145$. Before, we rigidly concluded that coin *A* was more likely. Now, since coin *A* is more likely to generate seven heads in a sequence of ten coin flips, we should assign a larger responsibility to coin *A* than to coin *B*. One possible way to assign these responsibilities is given by the formulas:

$$\frac{\Pr(0.7|\theta_A)}{\Pr(0.7|\theta_A) + \Pr(0.7|\theta_B)} = \frac{0.00179}{0.00179 + 0.00145} \approx 0.55$$

$$\frac{\Pr(0.7|\theta_B)}{\Pr(0.7|\theta_A) + \Pr(0.7|\theta_B)} = \frac{0.00145}{0.00179 + 0.00145} \approx 0.45$$

Making Soft Decisions in Coin Flipping | Step 3

As a result, instead of a vector *HiddenVector*, we now have a 2 × 5 responsibility profile *HiddenMatrix* that can be constructed from *Data* and *Parameters* (see figure below):

(*Data*, ?, *Parameters*) → *HiddenMatrix*

We call this transition the **E-step**.

Figure: Computing *HiddenMatrix* from *Data* = (0.4, 0.9, 0.8, 0.3, 0.7) and *Parameters* = (0.6, 0.82).

Expectation maximization: the M-step

When making hard assignments, we computed *Parameters* from *Data* and *HiddenVector* as follows:

$$\theta_A = \frac{HiddenVector \cdot Data}{HiddenVector \cdot \vec{1}}$$

$$\theta_B = \frac{(\vec{1} - HiddenVector) \cdot Data}{(\vec{1} - HiddenVector) \cdot \vec{1}}$$

To make soft assignments, note that the assignment of outcomes to the two coins can be represented by the binary responsibility matrix below. An occurrence of 1 in the *i*-th position of the first row means that we conclude that coin *A* generated the *i*-th sequence of flips, and an occurrence of 1 in the second row means that we conclude that coin *B* generated the *i*-th sequence of flips:

$$HiddenMatrix = \begin{matrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{matrix}$$

Thus, the first row of *HiddenMatrix*, denoted $HiddenMatrix_A$, is just *HiddenVector*, and the second row of *HiddenMatrix*, denoted $HiddenMatrix_B$, is just $\vec{1} - HiddenVector$. We can therefore rewrite the previous formulas for θ_A and θ_B in terms of *HiddenMatrix*:

$$\theta_A = \frac{HiddenMatrix_A \cdot Data}{HiddenMatrix_A \cdot \vec{1}}$$

$$\theta_B = \frac{HiddenMatrix_B \cdot Data}{HiddenMatrix_B \cdot \vec{1}}$$

Making Soft Decisions in Coin Flipping | Step 5

For the responsibility matrix in the figure below, we can now recompute *Parameters* as follows:

$$\theta_A = \frac{0.97 \cdot 0.4 + 0.12 \cdot 0.9 + 0.29 \cdot 0.8 + 0.99 \cdot 0.3 + 0.55 \cdot 0.7}{0.97 + 0.12 + 0.29 + 0.99 + 0.55} = \frac{1.41}{2.92} \approx 0.483$$

$$\theta_B = \frac{0.03 \cdot 0.4 + 0.88 \cdot 0.9 + 0.71 \cdot 0.8 + 0.01 \cdot 0.3 + 0.45 \cdot 0.7}{0.03 + 0.88 + 0.71 + 0.01 + 0.45} = \frac{1.69}{2.08} \approx 0.813$$

STOP and Think: Notice that the soft parameter choices $\theta_A = 0.483$ and $\theta_B = 0.813$ are a little closer to each other than the hard parameter choices $\theta_A = 0.467$ and $\theta_B = 0.85$. Why do you think that this is the case?

Making Soft Decisions in Coin Flipping | Step 6

In general, the transition

$$(\textit{Data}, \textit{HiddenMatrix}, ?) \rightarrow \textit{Parameters}$$

is called the **M-step**.

The expectation maximization algorithm

The **expectation maximization algorithm** starts with a random choice of *Parameters*. It then alternates between the E-step, in which we compute a responsibility matrix *HiddenMatrix* for *Data* given *Parameters*:

$$(Data, ?, Parameters) \rightarrow HiddenMatrix$$

and the M-step, in which we re-estimate *Parameters* using *HiddenMatrix*:

$$(Data, HiddenMatrix, ?) \rightarrow Parameters$$

Exercise Break: Carry out a few more steps of the expectation maximization algorithm for the data in the figure on the previous step. When should we stop the algorithm?

Applying expectation maximization to clustering

We are now ready to use the expectation maximization algorithm to modify the Lloyd algorithm into a **soft *k*-means clustering algorithm**. This algorithm starts from randomly chosen centers and iterates the following two steps:

- **Centers to Soft Clusters (E-step):** After centers have been selected, assign each data point a “responsibility” value for each cluster, where higher values correspond to stronger cluster membership.
- **Soft Clusters to Centers (M-step):** After data points have been assigned to soft clusters, compute new centers.

Centers to soft clusters

We begin with the “Centers to Soft Clusters” step. We have already used the term “center of gravity” when computing centers; if we think about the centers as stars and the data points as planets, then the closer a point is to a center, the stronger that center’s “pull” should be on the point. Given *k* centers *Centers* = (x_1, ..., x_k) and n points *Data* = ($Data_1$, ... , $Data_n$), we therefore need to construct a *k* × *n* responsibility matrix *HiddenMatrix* for which $HiddenMatrix_{i,j}$ is the pull of center *i* on data point *j*. This pull can be computed according to the Newtonian inverse-square law of gravitation,

$$HiddenMatrix_{i,j} = \frac{1/d(Data_j, x_i)^2}{\sum_{\text{all centers } i} 1/d(Data_j, x_i)^2}.$$

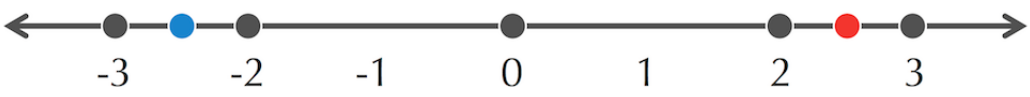
Unfortunately for Newton fans, the following **partition function** from statistical physics often works better in practice:

$$HiddenMatrix_{i,j} = \frac{e^{-\beta \cdot d(Data_j, x_i)}}{\sum_{\text{all centers } i} e^{-\beta \cdot d(Data_j, x_i)}}.$$

In this formula, e is the base of the natural logarithm ($e \approx 2.72$), and β is a parameter reflecting the amount of flexibility in our soft assignment and called — appropriately enough — the **stiffness parameter**.

The figure below illustrates different approaches for computing *HiddenMatrix* when *Data* represents points in one-dimensional space.

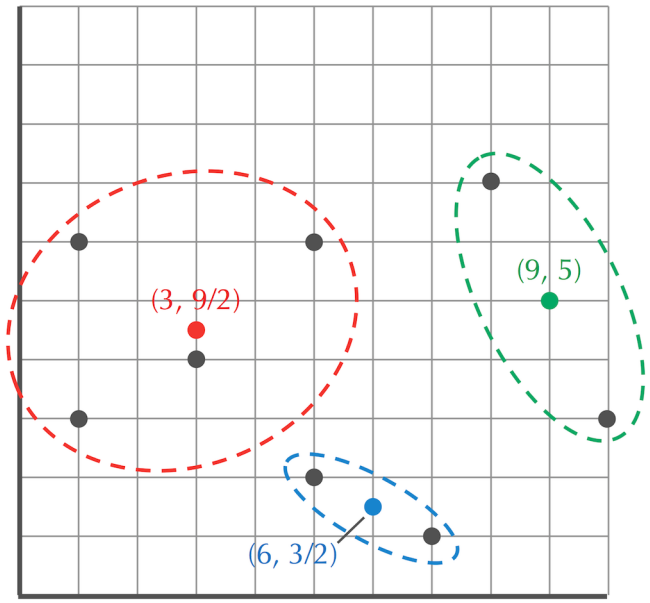
STOP and Think: How does the assignment of the points in Figure 1.21 to soft clusters change as $\beta \to \infty$ or as $\beta \to -\infty$? What about when $\beta = 0$?



0.992	0.988	0.500	0.012	0.008
0.008	0.012	0.500	0.988	0.992
0.924	0.881	0.500	0.119	0.076
0.076	0.119	0.500	0.881	0.924
0.993	0.982	0.500	0.118	0.007
0.007	0.018	0.500	0.982	0.993

Figure: (Top) Five one-dimensional points *Data* = (-3, -2, 0, +2, +3) with two centers (shown in blue and red) *Centers* = (-2.5, +2.5). (Bottom) Three versions of *HiddenMatrix* constructed for *Data* and *Centers*, using the Newtonian inverse-square law (first matrix) and the partition function with stiffness β = 0.5 (second matrix), and β = 1 (third matrix).

Exercise Break: Compute *HiddenMatrix* using the Newtonian inverse-square law for the three centers and eight data points shown in the figure below.



Soft clusters to centers

When we implemented the M-step for coin flipping, we obtained the following formulas for θ_A and θ_B:

$$\theta_A = \frac{HiddenMatrix_A \cdot Data}{HiddenMatrix_A \cdot \vec{1}}$$

$$\theta_B = \frac{HiddenMatrix_B \cdot Data}{HiddenMatrix_B \cdot \vec{1}}$$

In soft *k*-means clustering, if we let $HiddenMatrix_i$ denote the *i*-th row of *HiddenMatrix*, then we can update center x_i using the following analogue of the above formulas:

$$x_i = \frac{HiddenMatrix_i \cdot Data}{HiddenMatrix_i \cdot \vec{1}}$$

The updated center x_i is called the weighted center of gravity of the points *Data*. In practical terms, the influence that the point $Data_j$ has when updating center x_i is equal to the proportion of the pull between these two points with respect to the pull between x_i and all other data points.

HiddenMatrix is reproduced below:

0.992	0.988	0.500	0.012	0.008
0.008	0.012	0.500	0.988	0.992

0.924	0.881	0.500	0.119	0.076
0.076	0.119	0.500	0.881	0.924

0.993	0.982	0.500	0.118	0.007
0.007	0.018	0.500	0.982	0.993

Computing weighted centers of gravity for *HiddenMatrix* produces the following updated centers:

$$x_1 = \frac{0.993 \cdot (-3) + 0.982 \cdot (-2) + 0.500 \cdot (0) + 0.018 \cdot (2) + 0.007 \cdot (3)}{0.993 + 0.982 + 0.500 + 0.018 + 0.007} = -1.955$$

$$x_2 = \frac{0.007 \cdot (-3) + 0.018 \cdot (-2) + 0.500 \cdot (0) + 0.982 \cdot (2) + 0.993 \cdot (3)}{0.007 + 0.018 + 0.500 + 0.982 + 0.993} = 1.955$$

CODE CHALLENGE: Implement the expectation maximization algorithm for soft *k*-means clustering.

Input: Integers *k* and *m*, followed by a stiffness parameter *β*, followed by a set of points *Data* in *m*-dimensional space.

Output: A set *Centers* consisting of *k* points (*centers*) resulting from applying the expectation maximization algorithm for soft *k*-means clustering. Select the first *k* points from *Data* as the first centers for the algorithm and run the algorithm for 100 E-steps and 100 M-steps. Results should be accurate up to three decimal places.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/clustering/SoftKMeans.txt)

Sample Input:

```
2 2
2.7
1.3 1.1
1.3 0.2
0.6 2.8
3.0 3.2
1.2 0.7
1.4 1.6
1.2 1.0
1.2 1.1
0.6 1.5
1.8 2.6
1.2 1.3
1.2 1.0
0.0 1.9
```

Sample Output:

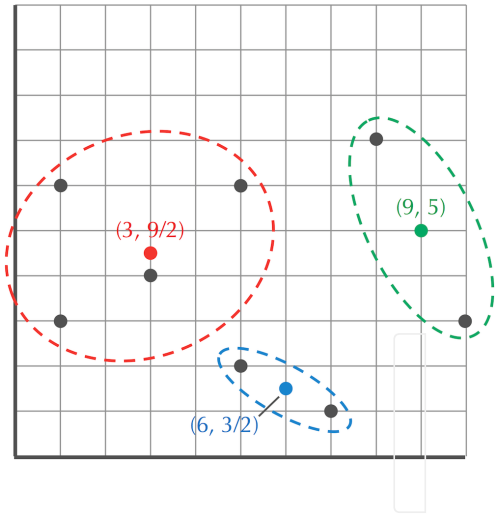
```
1.662 2.623
1.075 1.148
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10933/step/7

STOP and Think: Does the soft *k*-means clustering algorithm terminate? If not, how would you modify it to ensure that it does not run forever?

Exercise Break: A few steps ago, you computed *HiddenMatrix* for the eight points and three centers below. Now, recompute centers for these data points.



Exercise Break: Apply soft *k*-means clustering to the abridged yeast diauxic shift gene expression data, and compare the results with those of the Lloyd algorithm.

Abridged Diauxic Shift Dataset (http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/230genes_log_expression.txt)

Introduction to distance-based clustering

In a previous chapter, we discussed two types of approaches to evolutionary tree reconstruction with different strengths and weaknesses: distance-based algorithms (including the neighbor-joining algorithm), and alignment-based algorithms (including the Sankoff algorithm for the Small Parsimony Problem). Similarly, biologists do not always analyze the $n \times m$ gene expression matrix directly. Instead, they sometimes first transform this matrix into an $n \times n$ **distance matrix** D, where $D_{i,j}$ indicates the distance between the expression vectors for genes i and j (se figure below). In this section, we will see how to use a distance matrix to partition genes into clusters (see DETOUR: Transforming an Expression Matrix into a Distance/Similarity Matrix (https://stepic.org/lesson/Detour-Gene-Expression-Matrix-to-a-DistanceSimilarity-Matrix-10940) for more details).

	1 hr	2 hr	3 hr
g_1	10.0	8.0	10.0
g_2	10.0	0.0	9.0
g_3	4.0	8.5	3.0
g_4	9.5	0.5	8.5
g_5	4.5	8.5	2.5
g_6	10.5	9.0	12.0
g_7	5.0	8.5	11.0
g_8	3.7	8.7	2.0
g_9	9.7	2.0	9.0
g_{10}	10.2	1.0	9.2

	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	9.3	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	0.9	12.0	9.5	10.1	12.8	2.0	1.0
g_3	9.2	12.0	0.0	11.2	0.7	11.1	8.1	1.1	10.5	11.5
g_4	7.7	0.9	11.2	0.0	11.2	9.2	9.5	12.0	1.6	1.1
g_5	9.3	12.0	0.7	11.2	0.0	11.2	8.5	1.0	10.6	11.6
g_6	2.3	9.5	11.1	9.2	11.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	8.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.1	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	10.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	11.6	8.5	9.3	12.4	1.1	0.0

Figure: (Top) A toy gene expression matrix of ten genes measured at three time points. (Bottom) The corresponding distance matrix based on Euclidean distance.

In previous sections, we assumed that we were working with a fixed number of clusters *k*. But in practice, clusters often have subclusters, which have subsubclusters, and so on. To capture this cluster stratification, the hierarchical clustering algorithm uses an *n* × *n* distance matrix *D* to organize *n* data points into a tree (see figure below).

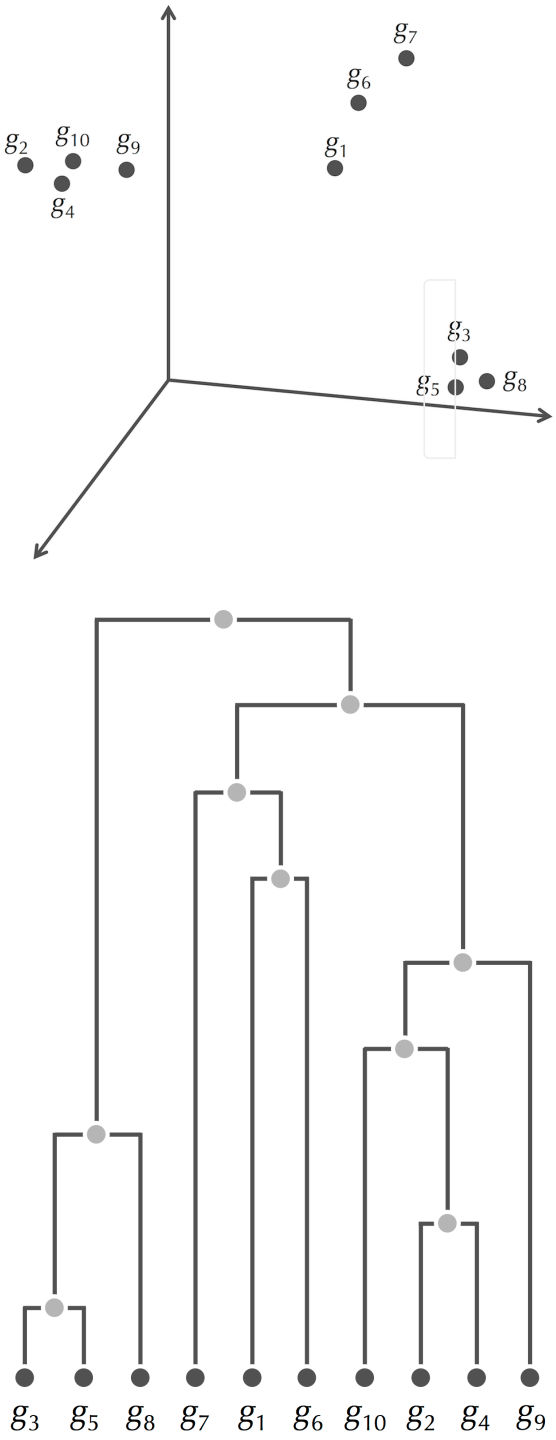


Figure: (Top) The gene expression vectors from the previous step as points in three-dimensional space. (Bottom) The tree produced from the distance matrix by the hierarchical clustering algorithm. Darker nodes in this tree are leaves corresponding to genes; lighter nodes are internal nodes.

As shown in the figure below, a horizontal line crossing the tree in *i* places divides the *n* genes into *i* clusters.

Exercise Break: The figure below illustrates two ways of clustering the data from the figure on the previous step. Find the remaining eight ways of clustering these data using the same tree.

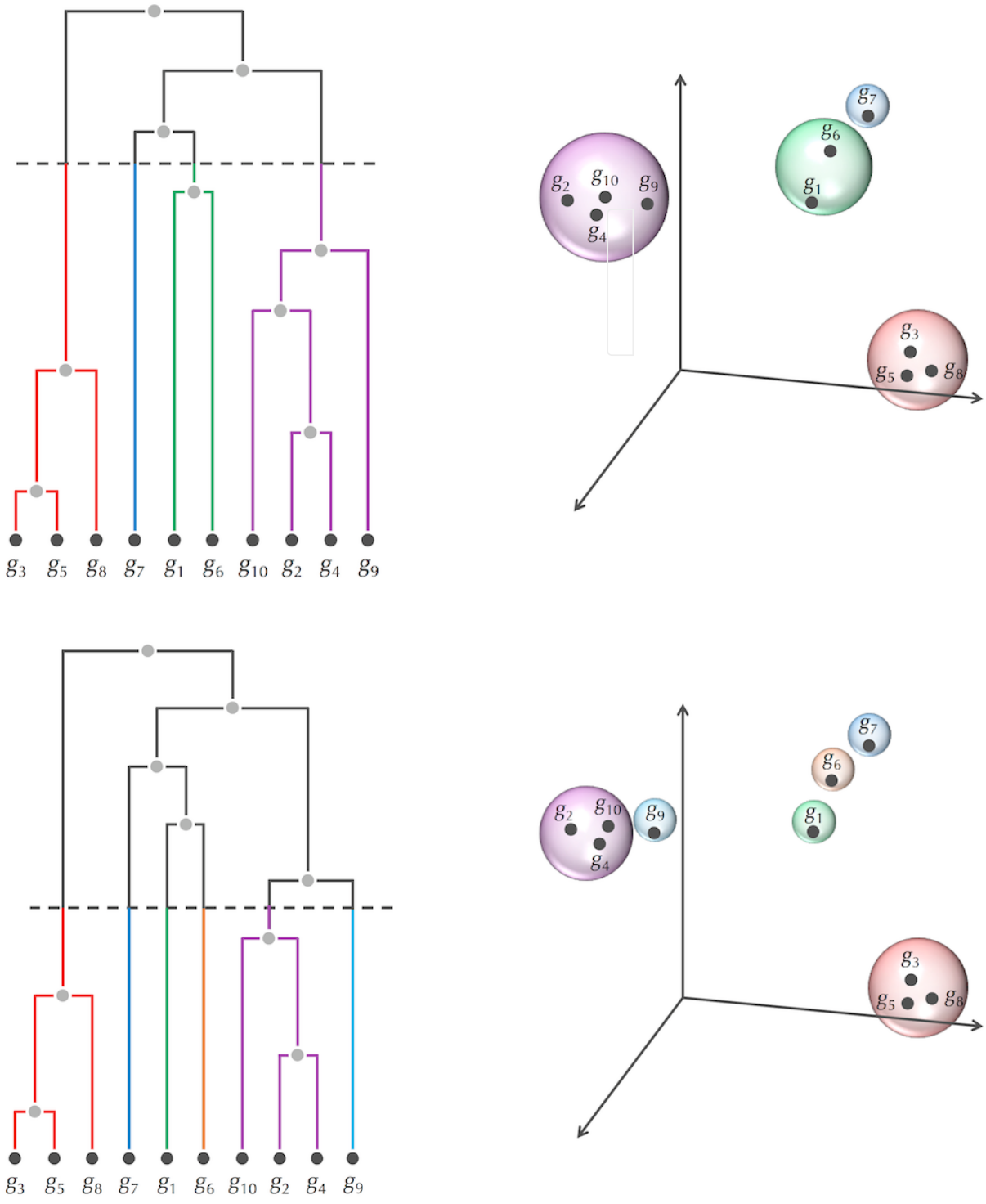


Figure: A tree with *n* leaves imposes *n* different ways of partitioning the data into clusters. (Top) The horizontal line through the tree (left) crosses the data in four places and partitions the data into four clusters (right). (Bottom) The same tree with a different horizontal line (left) partitions the data into six clusters (right).

Inferring clusters from a tree

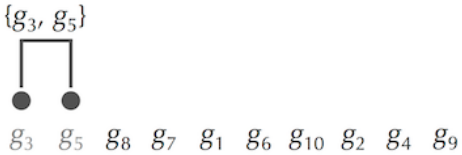
The question remains, however: how does the hierarchical clustering algorithm construct the tree from the distance matrix? **HierarchicalClustering**, whose pseudocode is shown below, progressively generates n different partitions of the underlying data into clusters, all represented by a tree in which each node is labeled by a cluster of genes. The first partition has n single-element clusters represented by the leaves of the tree, with each element forming its own cluster. The second partition merges the two "closest" clusters into a single cluster consisting of two elements. In general, the i-th partition merges the two closest clusters from the (i - 1)-th partition and has n - i + 1 clusters. We hope this algorithm looks familiar — it is **UPGMA** (from the chapter on evolutionary tree constuction) in disguise.

HierarchicalClustering(D, n)
  Clusters ← n single-element clusters labeled 1, ... , n
  construct a graph T with n isolated nodes labeled by single elements 1, ... , n
  while there is more than one cluster
    find the two closest clusters C_i and C_j
    merge C_i and C_j into a new cluster C_{new} with $|C_i| + |C_j|$ elements
    add a new node labeled by cluster C_{new} to T
    connect node C_{new} to C_i and C_j by directed edges
    remove the rows and columns of D corresponding to C_i and C_j
    remove C_i and C_j from *Clusters*
    add a row/column to D for C_{new} by computing $D(C_{new}, C)$ for each C in *Clusters*
    add C_{new} to *Clusters*
assign root in T as a node with no incoming edges
return T

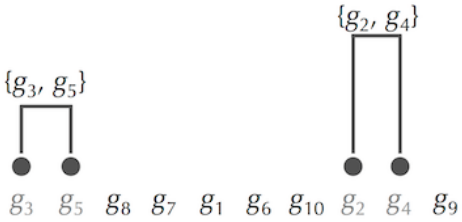
We have not yet defined how **HierarchicalClustering** computes the distance $D(C_{new}, C)$ between a newly formed cluster C_{new} and each old cluster C. In practice, clustering algorithms vary in how they compute these distances, with results that can vary greatly. One commonly used approach (see figure below) defines the distance between clusters C_1 and C_2 as the smallest distance between any pair of elements from these clusters,

$$D_{min}(C_1,C_2) = \min_{\text{all points } i \text{ in cluster } C_1,\ \text{all points } j \text{ in cluster } C_2} D_{i,j}\ .$$

	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	9.3	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	0.9	12.0	9.5	10.1	12.8	2.0	1.0
g_3	9.2	12.0	0.0	11.2	**0.7**	11.1	8.1	1.1	10.5	11.5
g_4	7.7	0.9	11.2	0.0	11.2	9.2	9.5	12.0	1.6	1.1
g_5	9.3	12.0	0.7	11.2	0.0	11.2	8.5	1.0	10.6	11.6
g_6	2.3	9.5	11.1	9.2	11.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	8.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.1	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	10.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	11.6	8.5	9.3	12.4	1.1	0.0



	g_1	g_2	g_3, g_5	g_4	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	**0.9**	9.5	10.1	12.8	2.0	1.0
g_3, g_5	9.2	12.0	0.0	11.2	11.1	8.1	1.0	10.5	11.5
g_4	7.7	0.9	11.2	0.0	9.2	9.5	12.0	1.6	1.1
g_6	2.3	9.5	11.1	9.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.0	12.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	8.5	9.3	12.4	1.1	0.0



	g_1	g_2, g_4	g_3, g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	7.7	9.2	2.3	5.1	10.2	6.1	7.0
g_2, g_4	7.7	0.0	11.2	9.2	9.5	12.0	1.6	1.0
g_3, g_5	9.2	11.2	0.0	11.1	8.1	**1.0**	10.5	11.5
g_6	2.3	9.2	11.1	0.0	5.6	12.1	7.7	8.5
g_7	5.1	9.5	8.1	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	1.6	10.5	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	8.5	9.3	12.4	1.1	0.0

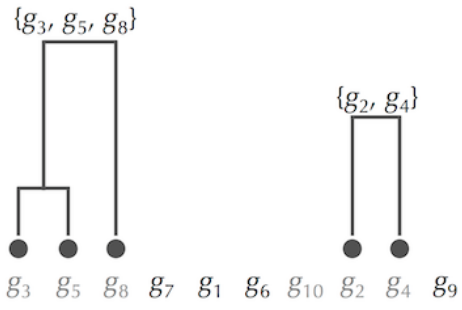


Figure: HierarchicalClustering in action. (Top left) The distance matrix from the figure on the next step (top left), with its minimum element shown in red, corresponding to genes g_3 and g_5. (Top right) Merging the single-element clusters containing g_3 and g_5. (Middle left) The updated distance matrix after computing D_{min} for the new cluster with respect to each other (single-element) cluster, with its minimum element shown in red. (Middle right) Merging the two clusters corresponding to the minimum element. (Bottom) Updating the distance matrix (left) and merging two additional clusters (right). Subsequent steps will reconstruct the tree from the figure on the next step (bottom right).

The distance function that we encountered with **UPGMA** uses the average distance between elements in two clusters,

$$D_{\text{avg}}(C_1, C_2) = \frac{\sum_{\text{all points } i \text{ in cluster } C_1} \sum_{\text{all points } j \text{ in cluster } C_2} D_{i,j}}{|C_1| \cdot |C_2|}$$

CODE CHALLENGE: Implement **HierarchicalClustering**.

Input: An integer *n*, followed by an *n* x *n* distance matrix.

Output: The result of applying **HierarchicalClustering** to this distance matrix (using D_{avg}), with each newly created cluster listed on each line.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/clustering/HierarchicalClustering.txt)

Sample Input:

```
7
0.00 0.74 0.85 0.54 0.83 0.92 0.89
0.74 0.00 1.59 1.35 1.20 1.48 1.55
0.85 1.59 0.00 0.63 1.13 0.69 0.73
0.54 1.35 0.63 0.00 0.66 0.43 0.88
0.83 1.20 1.13 0.66 0.00 0.72 0.55
0.92 1.48 0.69 0.43 0.72 0.00 0.80
0.89 1.55 0.73 0.88 0.55 0.80 0.00
```

Sample Output:

```
4 6
5 7
3 4 6
1 2
5 7 3 4 6
1 2 5 7 3 4 6
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-10934/step/7

Exercise Break: Apply **HierarchicalClustering** to the distance matrix in the figure below using D_{avg} instead of D_{min}.

Exercise Break: Apply **HierarchicalClustering** (with D_{avg}) to the 230-gene yeast dataset, and partition this dataset into six clusters. Do you expect these clusters to be roughly the same as the clusters shown in the figures below? If not, should we be concerned?

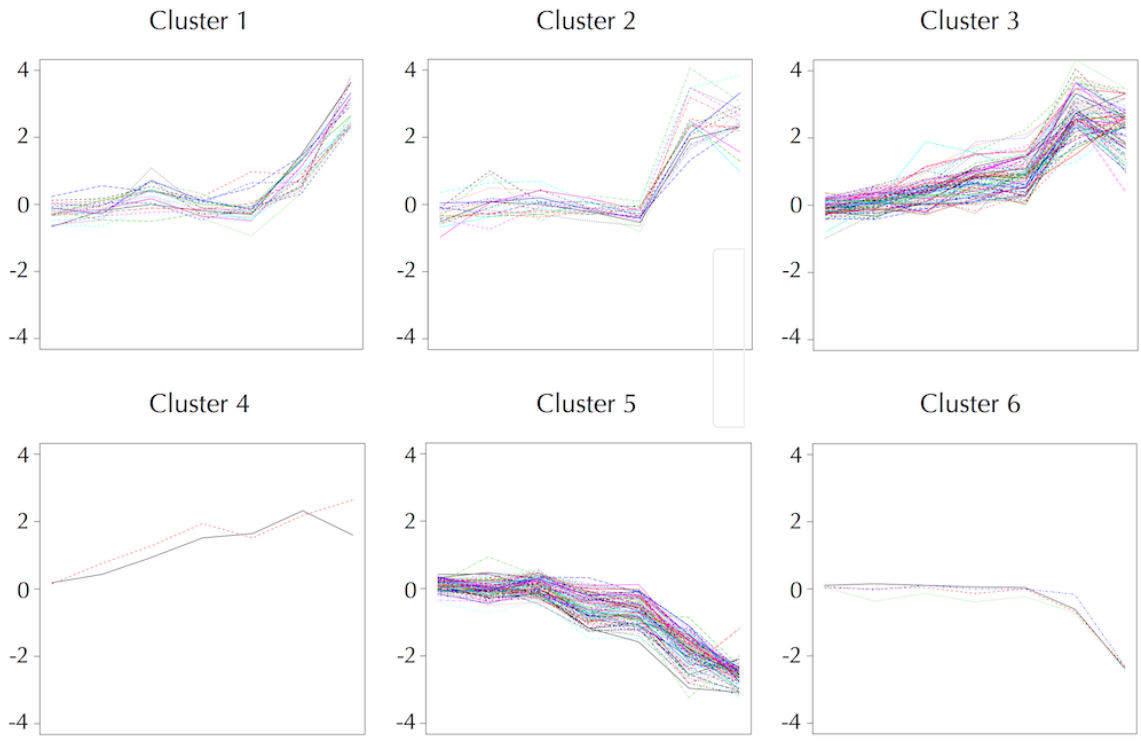
Abridged Diauxic Shift Dataset

(http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/230genes_log_expression.txt)

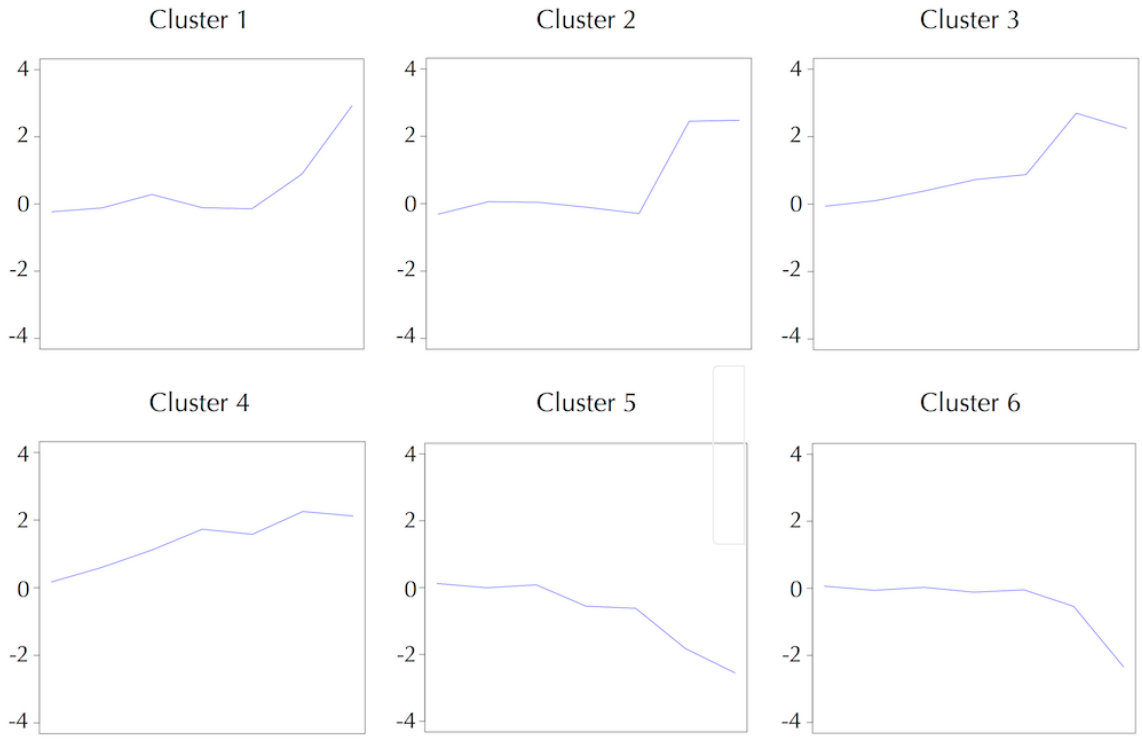
	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	g_9	g_{10}
g_1	0.0	8.1	9.2	7.7	9.3	2.3	5.1	10.2	6.1	7.0
g_2	8.1	0.0	12.0	0.9	12.0	9.5	10.1	12.8	2.0	1.0
g_3	9.2	12.0	0.0	11.2	0.7	11.1	8.1	1.1	10.5	11.5
g_4	7.7	0.9	11.2	0.0	11.2	9.2	9.5	12.0	1.6	1.1
g_5	9.3	12.0	0.7	11.2	0.0	11.2	8.5	1.0	10.6	11.6
g_6	2.3	9.5	11.1	9.2	11.2	0.0	5.6	12.1	7.7	8.5
g_7	5.1	10.1	8.1	9.5	8.5	5.6	0.0	9.1	8.3	9.3
g_8	10.2	12.8	1.1	12.0	1.0	12.1	9.1	0.0	11.4	12.4
g_9	6.1	2.0	10.5	1.6	10.6	7.7	8.3	11.4	0.0	1.1
g_{10}	7.0	1.0	11.5	1.1	11.6	8.5	9.3	12.4	1.1	0.0

Analyzing the diauxic shift with hierarchical clustering

The figure below visualizes expression vectors for each of the six clusters obtained after applying **HierarchicalClustering** (using D_{avg}) to the yeast dataset.



Their averages are shown below.



Exercise Break: Implement **HierarchicalClustering** (with D_{min} rather than D_{avg}) and apply it to partition the yeast gene expression dataset into six clusters. How does the result differ from the figure on the previous step?

STOP and Think: HierarchicalClustering and the Lloyd algorithm (see the figure below) have produced different clusters. Should we be concerned?

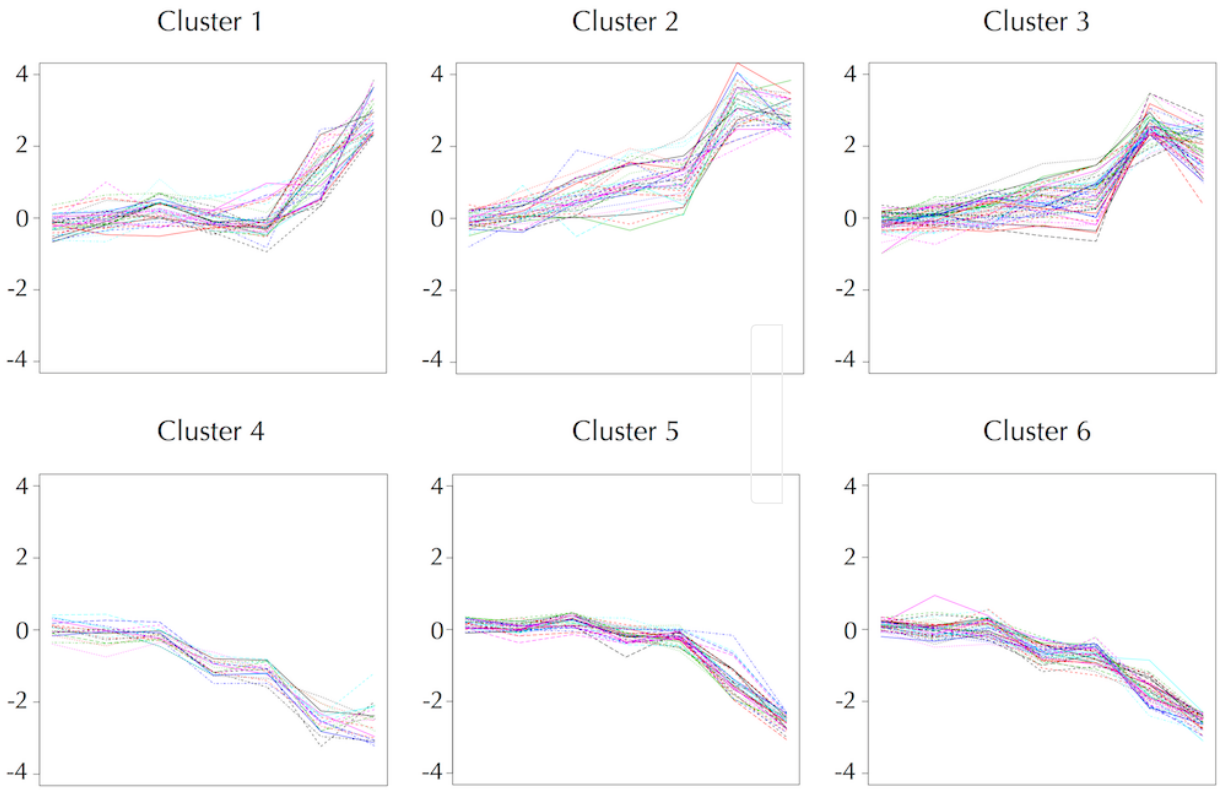


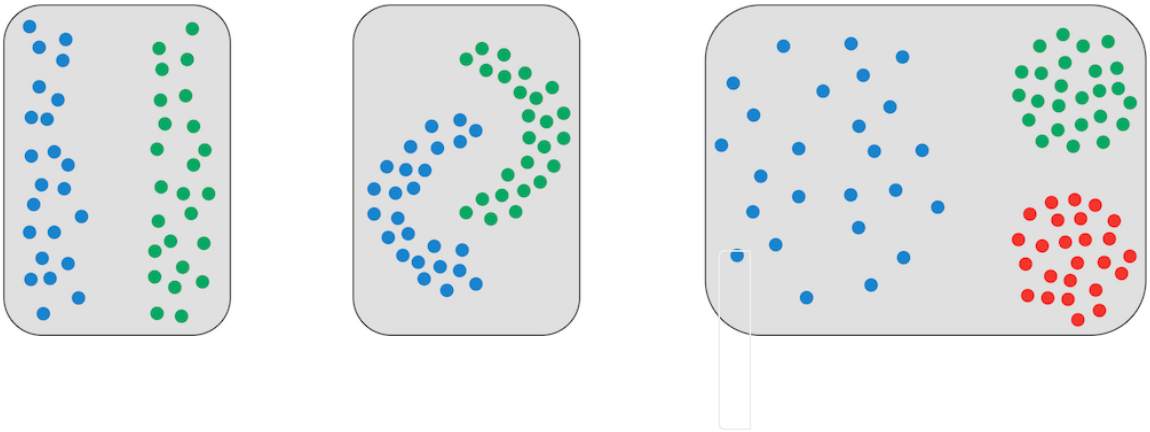
Figure: The six yeast gene clusters produced by the Lloyd algorithm. Note how they differ from the six clusters produced by **HierarchicalClustering** from the previous step.

Biologists are not discouraged by the fact that different clustering approaches may produce different clusters because applying these clustering algorithms is often just the first step on the road to discovery (see DETOUR: Clustering and Corrupted Cliques (https://stepic.org/lesson/Detour-Clustering-and-Corrupted-Cliques-10941) for yet another clustering approach). For this reason, gene expression studies are typically followed by experimental work to confirm that derived clusters make sense biologically. Once clusters have been generated, further research often focuses on specific genes within these clusters.

For example, each of the yeast clusters produced by **HierarchicalClustering** can be further analyzed to reveal sub-clusters of genes with even more pronounced expression profiles than the genes in the entire cluster. In particular, cluster 1 contains seven genes exhibiting rather slight changes during the first six checkpoints and a surge in gene expression at the final checkpoint. Biologists discovered that six of these seven genes have the **carbon source response element (CSRE) regulatory motif** with consensus sequence "CATTCATCCG" in their upstream regions. Further analysis of the entire yeast genome revealed that only four other yeast genes have this motif in their upstream region, suggesting that it was a good idea to group these six genes into a sub-cluster within cluster 1.

The more important issue, however, is to understand why these six genes are related. Yeast prefers using glucose as an energy source compared to other compounds like ethanol, and so in the presence of glucose, the transcription of genes responsible for metabolizing these less tasty compounds is repressed. Researchers have thus concluded that the CSRE motif somehow helps yeast sense the presence of glucose and activates the six genes in question when the organism runs out of glucose, thus serving as an important component of the diauxic shift.

Finally, if you believe that we have exhausted every possible avenue of clustering, take another look at the figure below; none of the clustering algorithms we have encountered can produce these clusters. Clustering seems like a straightforward problem in part because the human eye is so adept at grouping points into shapes. After all, computer vision researchers are still trying to teach computers to mimic our visual experience of the world, which is the outcome of millions of years of evolution.



As we mentioned earlier, gene expression analysis has a wide variety of applications, including cancer studies. In 1999, Uri Alon analyzed gene expression data for 2,000 genes taken from 40 colon tumor tissues and compared them with data taken from colon tissues belonging to 21 healthy individuals. We can represent his data as a 2,000 × 61 gene expression matrix, where the first 40 columns describe tumor samples and the last 20 columns describe normal samples.

Now, suppose you performed a gene expression experiment with a colon sample from a new patient, corresponding to a 62nd column in an augmented gene expression matrix. Your goal is to predict whether this patient has a colon tumor. Since the partition of tissues into two clusters (tumor vs. healthy) is known in advance, it may seem that classifying the sample from a new patient is easy. Indeed, since each patient corresponds to a point in 2,000-dimensional space, we can compute the center of gravity of these points for the tumor sample and for the healthy sample. Afterwards, we can simply check which of the two centers of gravity is closer to the new tissue.

Alternatively, we could perform a blind analysis, pretending that we do not already know the classification of samples into cancerous vs. healthy, and analyze the resulting 2,000 x 62 expression matrix to divide the 62 samples into two clusters. If we obtain a cluster consisting predominantly of cancer tissues, this cluster may help us diagnose colon cancer.

FINAL CHALLENGE: These approaches may seem straightforward, but it is unlikely that either of them will reliably diagnose the new patient. Why do you think this is? Given Alon's 2,000 × 61 gene expression matrix and gene data from a new patient, derive a superior approach to evaluate whether this patient is likely to have a colon tumor.

40 Cancer Samples (http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/colon_cancer.txt)

21 Healthy Samples (http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/colon_healthy.txt)

Unknown Sample (http://bioinformaticsalgorithms.com/data/realdatasets/Clustering/colon_test.txt)

There are many ways to quantify the similarity between expression vectors $x = (x_1,...,x_m)$ and $y = (y_1,...,y_m)$. One possibility is the dot product, $\sum_{i=1}^{m} x_i \cdot y_i$. Another is the **Pearson correlation coefficient** *PearsonCorrelation*(*x*, *y*), where

$$PearsonCorrelation(x, y) = \frac{\sum_{i=1}^{m}(x_i - \mu(x)) \cdot (y_i - \mu(y))}{\sqrt{\sum_{i=1}^{m}(x_i - \mu(x))^2 \cdot \sum_{i=1}^{m}(y_i - \mu(y))^2}}$$

In the above formula, μ(x) denotes the means of all coordinates of vector *x*.

STOP and Think: Given a vector *x*, which vectors y maximize and minimize *PearsonCorrelation*(*x*, *y*)?

The Pearson correlation coefficient varies between -1 and 1, where -1 indicates total negative correlation, 0 indicates no correlation, and 1 indicates total positive correlation. Based on the Pearson correlation coefficient, we can define the **Pearson distance** between vectors *x* and *y* as

$$PearsonDistance(x,y) = 1 - PearsonCorrelation(x,y)$$

Exercise Break: Compute the Pearson correlation coefficient for the following pairs of vectors:

1. (cos *α*, sin *α*) and (sin *α*,cos *α*) for an arbitrary value of *α*;
2. $(\sqrt{0.75}, 0.5)$ and $(-\sqrt{0.75}, 0.5)$

In expression analysis studies, a similarity matrix R is often transformed into a **similarity graph** $G(R, \theta)$. The nodes of this graph represent genes, and an edge connects genes i and j if and only if the similarity between them ($R_{i,j}$) exceeds a threshold value θ.

STOP and Think: Consider a clustering of genes that satisfies the Good Clustering Principle: the similarity between any two genes within the same cluster exceeds θ, and the similarity between any two genes in different clusters is less than θ. What does the similarity graph $G(R, \theta)$ look like for these genes?

If clusters satisfy the Good Clustering Principle, then there should be some value of θ such that each connected component of $G(R, \theta)$ is a **clique**, or a graph in which every pair of nodes are connected by an edge (see the figure below). In general, a graph whose connected components are all cliques is called a **clique graph**.

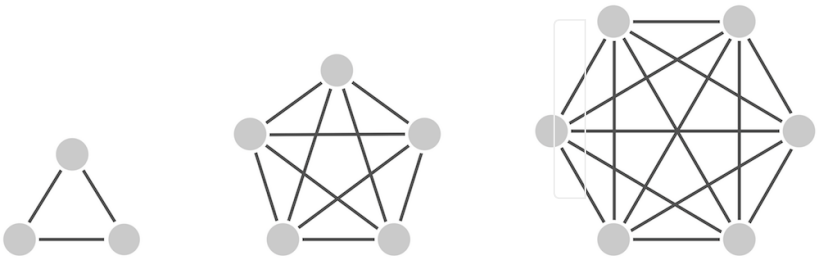


Figure: A clique graph consisting of three cliques.

Errors in expression data and the absence of a universal threshold θ often result in corrupted similarity graphs whose connected components are not cliques (see the figure below).

Either genes from the same cluster may have a similarity value falling below θ, thus removing edges from a clique, or genes from different clusters may have a similarity value exceeding θ, thus adding edges between different cliques. This observation leads us to ask how to transform a corrupted similarity graph into a clique graph using the smallest number of edge additions and deletions.

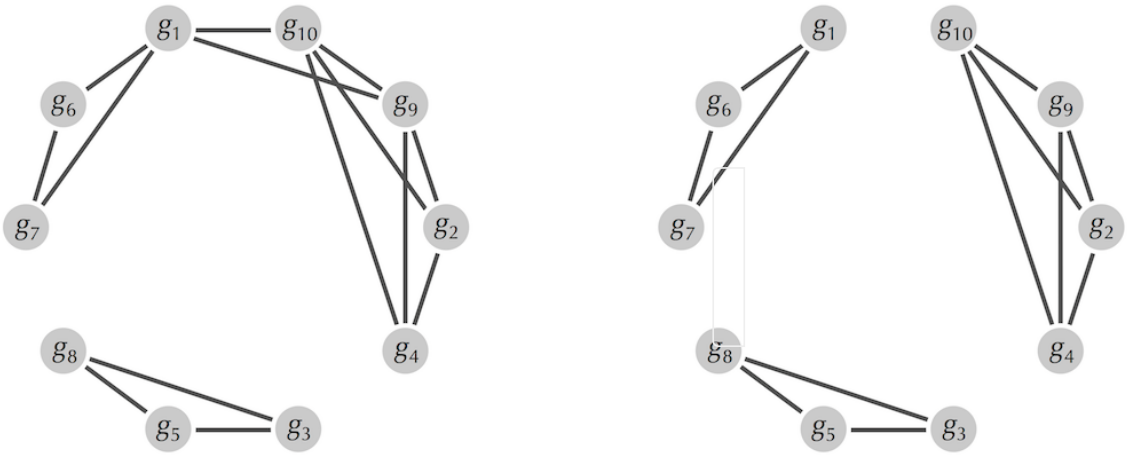


Figure: (Left) One possible similarity graph for the genes from the figure on the next step. (Right) The similarity graph can be transformed into a clique graph (right) by removing edges (g_1, g_{10}) and (g_1, g_9).

Corrupted Cliques Problem: *Find the minimum number of edges that need to be added or deleted to transform a graph into a clique graph.*

Input: A graph.

Output: The minimum number of edge additions and deletions that transform this graph into a clique graph.

The Corrupted Cliques Problem is difficult to solve exactly, so some heuristics have been proposed. The **Cluster Affinity Search Technique algorithm (CAST)**, described below, performs remarkably well at clustering gene expression data. Define the similarity between gene *i* and cluster *C* as the average similarity between *i* and all genes in *C*:

$$R_{i,\,C} = \sum_{\text{all elements } j \text{ in cluster } C} \frac{R_{i,j}}{|C|}$$

Given a threshold *θ*, a gene *i* is *θ*-close to cluster *C* if $R_{i,C}$ > θ and θ-**distant** from *C* otherwise. A cluster is called **consistent** if all genes in *C* are *θ*-close to *C* and all genes not in *C* are *θ*-distant from *C*. **CAST** uses the similarity graph *G* and the threshold θ to iteratively find consistent clusters by starting with a single-element cluster *C* and then adding the “closest” gene not in *C* and removing the “most distant” gene in *C*. After a consistent cluster is found, all nodes in cluster *C* are removed from the similarity graph, and **CAST** iterates over the resulting smaller graph.

CAST(*R*, *v*)
  Graph ← *G*(*R*, *v*)
  Clusters ← empty set
  while *Graph* is nonempty
    C ← a single-node cluster consisting of a node of maximal degree in *Graph*
    while there exists a v-close gene *i* not in *C* or a v-distant gene *i* in *C*
      find the nearest v-close gene *i* not in *C* and add it to *C*
      find the farthest v-distant gene *i* in *C* and remove it from *C*
  add *C* to the set *Clusters*
  remove the nodes of *C* from *Graph*
return Clusters

Exercise Break: Implement **CAST** and use it to cluster the abridged gene expression dataset.